2.2: Tech Stack

Pick the tools you'll use to build your MVP. Match the tool to the task and play to your strengths. It's totally fine to use languages you already know from class.

Answer all of the following for your tech stack:

- What language are you writing in?
- What libraries, packages, or external tools do you need? If you're writing Bash, which commands or utilities will you rely on **curl**, **grep**, **jq**, **cron**, **etc.**)? If you're writing Java, which external libraries (if any) do you need?
- How will you store data? For an MVP, a plain text / JSON / CSV file is often enough, so there's no need to jump to a full database.
- If you're pulling data from an external source, what is it and how will you access it? (See the APIs & External Data Sources post on Ed.)

Deliverable: A tech stack list in the design doc covering language, libraries/commands, data storage, data sources (if applicable), and anything else relating to the technical side of your project.

I am going to write in java. I will use the ArrayList, Sort, YearMonth function/methods, IO file, exceptions, GUI, probably inheritance, interface. I will store data in files, plain text files. No data from outside.

2.3: Chunked Build Plan

With your MVP scoped out and your tech stack picked, plan the actual build. Break the work into bite-sized milestones, where each chunk should roughly correspond to one meaningful Git commit. An example for a Discord bot would look like:

- Set up Git repo and environment
- Get the bot to log in and say "Hello"
- Write the web scraper function
- Connect the scraper data to the bot's messages

Yours should follow the same structure: small, testable, commit-sized steps that build on each other. You must produce between 5 and 10 steps. Fewer than 5 means your chunks are too big; more than 10 means they're too small. If you want feedback before

turning this in, go to PSO. TAs can help you iterate on scope, making sure your project isn't too big or too small, before you lock it in.

Deliverable: A chunked milestone list (between 5 and 10 steps).

Set up github repo and the necessary files
Write the first version that can rank tasks based on deadlines
Add the sorting function that includes importance as a factor
Add the sorting function that includes needed time as a factor
Add the function that user can change the ratio of each part
Add the function that schedules can be changed based on current progress.
Add the function of analyzing the efficiency, type spent time and needed time to calculate.